Documentation using OpenAPI and JSDoc

Note that the code referred to below begins in a state similar to the tag in the previous laboratories. You may wish to use git to check out this tag to follow along. Alternatively, you can try implementing the tasks on your own current code state if you wish.

Overview

Documenting both code and product features should be high on the list for any professional developer. In the case of APIs and the JavaScript stack, as usual, there are many options. However, two de-facto standards have emerged.

For **API documentation**, which covers your resource endpoint URIs, the allowed HTTP methods and status codes, and the structures of JSON request and response data, the most widely used approach is now the OpenAPI Specification. This was originally the Swagger Specification before SmartBear Software open-sourced it, and it is often used with Swagger tools. However, there are a wide range of well-developed alternative open-source tools that now produce and consume OpenAPI specs. The specification is a general-purpose syntax and allows development teams to share standardised documentation for APIs written in different languages.

For **code documentation**, which means providing explanatory doc strings inside your code files, there is less standardisation but JSDoc is widely used in the Node and JavaScript community and should be high on the list of packages to consider when documenting your own code.

Note: we are using OpenAPI/JSDoc for documentation of the blog, but you are free to explore and use alternatives for the blog and for your coursework. **What matters most is creating useful, flexible, explanatory documentation.** Other tools exist if you wish to use them.

This worksheet looks at these topics:

- ⇒ Setting up an OpenAPI Specification (OAS) document for your own API.
- ⇒ Adapting your previously defined JSON schemas so they can be re-used in your OpenAPI specification.
- ⇒ Using open source tools to read, format, and present the API docs in an interactive web interface, in our case hosted through Koa.
- ⇒ Configuring JSDoc to generate developer documentation for your application code.

OpenAPI Specification

Details on the OpenAPI specification options and its syntax are available at the following address and links inside: https://swagger.io/docs/specification/about/

Update Your JSON Schemas to actual JSON

Since we want to use our JSON Schema files from previous weeks with OpenAPI tools, we need to ensure they are valid JSON; previously we exported JavaScript objects from the schema files so these now need to be refactored.

1. Create a new **docs** folder to store the API documentation. Create a sub-folder "**yaml**" inside **docs** folder.
2. Duplicate (all) '*.schema.ts' file(s) from the **schemas** folder to docs/yaml folder.

VT6003CEM Web API Development (Hong Kong version)
Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

3. Rename each '*.schema.ts' file as '*.json'.
4. Remove the "export const xxx =" code from the beginning of each file.
5. Ensure all property names are in quotes, since properties in JSON are strings.
6. Remove any trailing commas ',' after sequences of nested objects.
    - For example {[1,2,3], [4,5,6],} would change to {[1,2,3], [4,5,6]}
7. Ensure each file is now valid JSON by linting it.

The following links may be helpful for the steps above:

- https://www.convertonline.io/convert/js-to-json - to convert property names to strings
- https://jsonlint.com/ - to check for any remaining issues such as trailing commas

Begin your OpenAPI Specification Document

OpenAPI specs can be written in JSON or in YAML markup. Here we use YAML. The basic structure of an OpenAPI document (OAS file) is summarised here: https://swagger.io/docs/specification/basic-structure/

⇒ Please review the linked page to get an overview of the YAML syntax and keywords used in the specification.

To begin documenting the API create a new schema called '**openapi.yaml**' in the **docs/yaml** directory. At the top of your OAS file put the following meta-information about your API:

```
openapi: 3.0.0
info:
  version: 1.0.0
  title: Back End Demonstration API
  description: A simple blog API.
  contact:
    name: [Your name]
    email: [Your e-mail]
  license:
    name: Apache 2.0
    url: 'https://www.apache.org/licenses/LICENSE-2.0.html'
servers:
  - url: 'https://[your API url] '
    description: Development server
                                                                    openapi.yaml
```

We give the version of the spec being used (v3 is the most recent) so that tools and scripts reading the document know how to parse it correctly.

Next we need to specify the groups of resources included in the API specification. This can be done by using the '**tags**' keyword, which gives a list of tags each of which defines a separate section in the resulting documentation. For this example, we add two tags:

```
tags:
  - name: Users
    description: API endpoints for user management.
  - name: Articles
    description: Access and perform CRUD operations on blog entries.
                                                                    openapi.yaml
```


Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

Now we get to the main purpose of the OAS document, which is defining the **API paths**, their HTTP **methods and status codes**, and the **data representations** they use. All of this information is placed under a new OAS keyword '**paths**' which has sections corresponding to each of the individual URIs defined by the files in your '**routes**' directory.

For example, consider the *articles* resource and its router '**routers/articles.ts**'. This route defines two paths: `/api/v1/articles` and `/api/v1/articles/:id`. The first path is associated to GET and POST methods, and the second path is associated to GET, PUT, and DELETE methods in the router file.

We begin the definition of the GET method for the second path in the OAS file as follows:

```
paths:
  /articles/{id}:
    get:
      tags:
        - Articles
      description:
        Get article by id.
    put:
      tags:
        - Articles
      description:
        Update article by ID.
    delete:
      tags:
        - Articles
      description:
        Delete article by ID.
```

openapi.yaml

This nests the URI path `/users/{id}` under the 'paths' section. Note that this path does NOT include the base address, which can be found in the URL property of the servers' section above.

Then, under the path we have a section for each of the HTTP methods a client can use in a request to that path. These sections have a brief description, and a list of the '**tags**' section(s) that the documentation for this path and method should appear in, when generated.

Next, we add key information that our API client(s) will be interested in: the possible HTTP response codes to each of the HTTP request methods, and a short description of what they mean in the context of our API.

Expand the code you have in the YAML file by adding the dark text as indicated below:

```
paths:
  /articles/{id}:
    get:
      tags:
        - Articles
```


Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

```
      description:
        Get article by id.
      responses:
        '200':
          description: View of an article record.
        '404':
          description: Article was not found in the DB.
    put:
      tags:
        - Articles
      description:
        Update article by ID.
      responses:
        '200':
          description: Update confirmation and link to record.
        '404':
          description: Article was not found in the DB.
    delete:
      tags:
        - Articles
      description:
        Delete article by ID.
      responses:
        '200':
          description: Delete confirmation.
        '404':
          description: User was not found in the DB.
                                                                    openapi.yaml
```

In addition, we should document the path parameter {id} and what it is for. To do so, add the following code as the first child block inside '/users/{id}':

```
  /articles/{id}:
    parameters:
      - in: path
        name: id
        schema:
          type: integer
        required: true
        description: Numeric id of the article
    get:
```

Preview the OpenAPI Specification Documentation

We now have enough of the OAS document to set up a dynamic HTML preview. We will use the extremely popular **redoc** package which is designed to read and format OAS documents for display in the browser: https://www.npmjs.com/package/redoc

At the root of the docs folder, add a new 'blogapi.html' file with the following content, which loads Redoc from a CDN and reads your 'openapi.yaml' file in to display:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Blog API Docs</title>
    <meta charset="utf-8"/>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link href="https://fonts.googleapis.com/css?family=Montserrat:300,400,700|Roboto:300,400,700" rel="stylesheet">
```


Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

```
    <style>
      body {
        margin: 0;
        padding: 0;
      }
    </style>
  </head>
  <body>
    <redoc spec-url='./yaml/openapi.yaml'></redoc>
    <script src="https://cdn.jsdelivr.net/npm/redoc@next/bundles/redoc.standalone.js"> </script>
  </body>
</html>
```

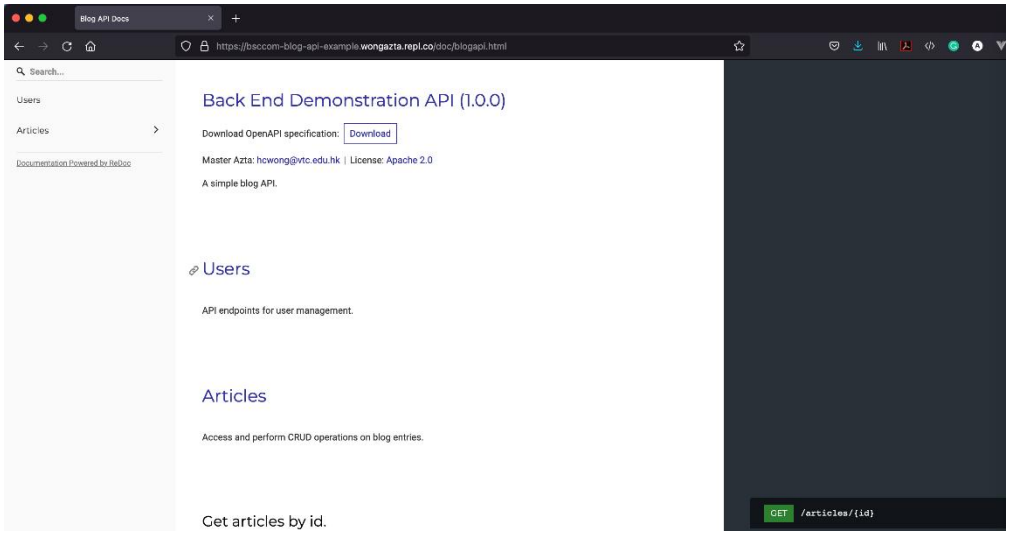
index.html

The key lines are the two lines inside the <body>: the <redoc> tag which specifies a path to your OAS document, and the <script> tag which actually loads Redoc to read the OAS. **You must ensure that the *spec-url* attribute value in the <redoc> tag correctly points to the relative location of your OpenAPI Specification document in your own project directory structure.**

Since the documentation is HTML-based we need to serve it over HTTP(S). It may do this via the Koa-static-route framework and add the following lines to the **index.ts** file:

```
// ...
import serve from 'koa-static';
// ...
app.use(serve('./docs'));
```

Now, you can try to browse the OpenAPI page via: https://[your-replit-url]/docs/blogapi.html



Include Request and Response Data in the OAS

Each of the path and method combinations documented in the OAS may consume and/or return JSON data, and we should use the OAS to *document the data structures involved*. This is where our JSON Schema definitions come in. We will refactor these definitions, and extend them, and use the

Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

OAS 'definitions' and '$ref' keywords to label and refer to parts of our JSON Schemas from the openapi.yaml document.

See this link for details of referencing other schemas from anywhere in a schema: https://json-schema.org/understanding-json-schema/structuring.html#reuse

First, open the 'articles.json' schema document. Create a new property called '**definitions**' directly after the '**$schema**' property. Inside this property, add a property called '**article**' whose schema contains the rest of the properties of the JSON Schema for a user that you wrote before:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "definitions": {
    "article": {
      ... [origin JSON schema]
    }
  }
}
```

articles.json

The schema at **article.json#/definitions.article** describes the data we want clients to send to the API when a user registers. Thus, we can use it in our OAS document to specify the required structure of the request body of a **POST** operation to create users. To do this we use the "**$ref**" pointer keyword described in the link provided above. Extend the 'post' section as follows:

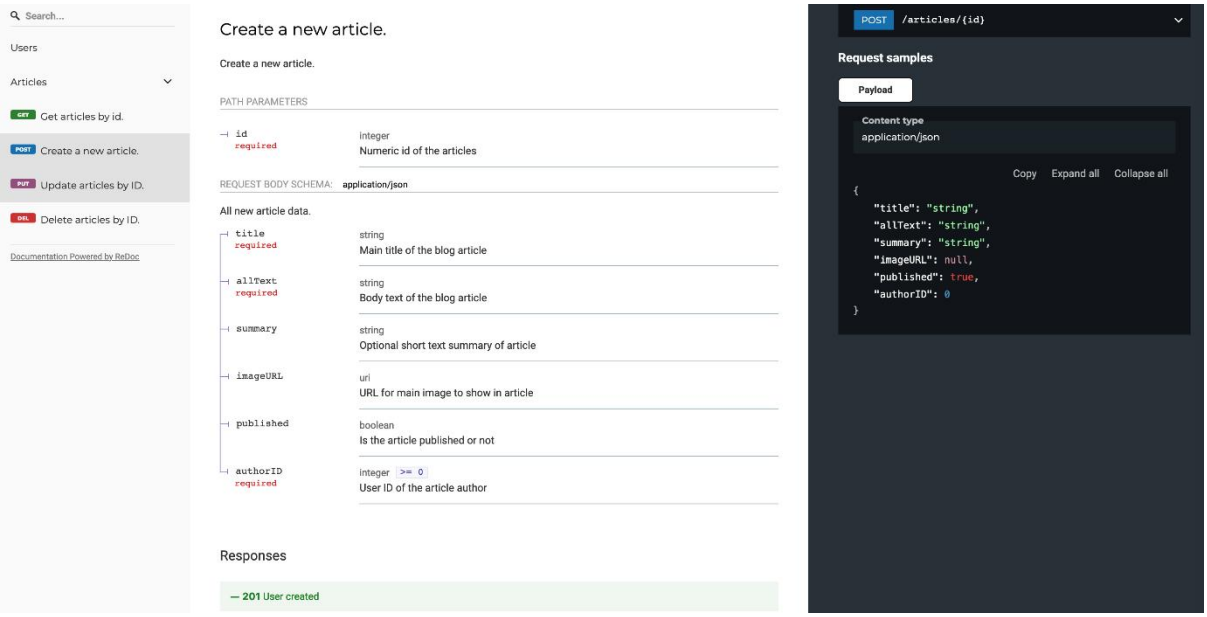
```
    post:
      tags:
        - Articles
      description:
        Create a new article.
      requestBody:
        description: All new article data.
        required: true
        content:
          application/json:
            schema:
              $ref: ./article.json#/definitions/article
      responses:
        '201':
          description: Articles created
```

openapi.yaml

Note that we added a 'requestBody' section, whose 'content' definition contains a pointer to the external user definition in the JSON Schema document.

VT6003CEM Web API Development (Hong Kong version)
Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

Refresh your ReDoc webpage and the documentation should now show the JSON schema information along with the HTTP methods and status codes:



Review materials for OpenAPI Specification and JSON Schemas.

So far you have just set up a very simple OAS, but you have been able to rely on using JSON Schemas that are also used for validation.

There are many OAS and JSON Schema keywords/tags that can be used, and you should thoroughly explore the relevant documentation for your Coursework, beginning with:

- ⇒ For OpenAPI Specification
  - https://app.swaggerhub.com/help/tutorials/openapi-3-tutorial
  - https://swagger.io/docs/specification/basic-structure/
  - https://github.com/OAI/OpenAPI-Specification/blob/master/versions/3.0.3.md
- ⇒ For JSON Schema
  - https://cswr.github.io/JsonSchema/spec/introduction/
  - https://json-schema.org/understanding-json-schema/basics.html
  - https://json-schema.org/understanding-json-schema/reference/index.html
  - https://json-schema.org/understanding-json-schema/structuring.html

There are also many tutorials online and a full 3-hour video course on OpenAPI-based API development in the *Developing Beyond Web API Development* course collection on LinkedIn Learning that is worth checking out:

https://www.linkedin.com/learning-login/share?forceAccount=true&redirect=https%3A%2F%2Fwww.linkedin.com%2Flearning%2Fcollections%2F6721666745116999680%3Ftrk%3Dshare_collection_url&account=2039756

VT6003CEM Web API Development (Hong Kong version)
Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk

Basic Worksheet Tasks

What you should do now

- ⇒ **Break for a while and start designing your coursework API NOW!!!**
- ⇒ **Begin documenting your Coursework API with an OpenAPI Specification file in JSON or YAML format.**

Advanced Worksheet Tasks

- ⇒ Assuming your API is operational, use Swagger Inspector (link below) or similar to *automatically document your API in the form of an OpenAPI Specification* by using it to send test requests and record the responses and status codes. You will have to read up on how the inspector works and will probably need to sign up for a free Swagger account. https://swagger.io/docs/swagger-inspector/how-to-create-an-openapi-definition-using-swagger/
- ⇒ See if you can use Swagger's tools to *automatically generate a functional pure JavaScript API* starting from just your OpenAPI Specification file (which must cover all the paths, and request/response JSON schemas needed to define your API in a fairly complete way). Run it on a server, plug it in to some mock data or your existing DB, and test its endpoints with Postman. Does this provide a fast route to API development?

VT6003CEM Web API Development (Hong Kong version)
Copyright to Vocational Training Council.
This document is for VTC students for their own use in completing their learning for this module and should not be passed to third parties or posted on any website. Any infringements of this rule should be reported to cw-it@vtc.edu.hk